Simulação e Teste de Software

Laboratório 9: Testes de Mutação

Prof. Luciano Rossi

Departamento de Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026

Laboratório 9

Contexto

Objetivo desta aula

Medir suítes com o mutmut e achar a causa de cada mutante vivo. Depois, separar os equivalentes e levar o escore ao máximo.

- Cada exercício é um projeto próprio, numa pasta `ex1` a `ex4`
- Registre os números pedidos antes e depois de cada melhoria
- A entrega segue como nos laboratórios anteriores

Por que o Codespaces

Contexto

- O mutmut 3.8.0 não roda no Windows nativo: ao iniciar, ele pede o WSL[1]
- O GitHub Codespaces abre, no navegador, um VS Code ligado a uma máquina Linux
- Assim todos usam o mesmo sistema, com as mesmas versões do gabarito

Como abrir

1. Crie um repositório `lab09` na sua conta do GitHub
2. No repositório, use *Code*, aba *Codespaces*, e crie um *codespace*
3. Use o terminal integrado do VS Code para os comandos a seguir

[1] *WSL (Windows Subsystem for Linux): camada do Windows que executa um Linux.*

Preparação do Projeto

Código

Terminal do Codespace

```
pip install pytest==9.1.1 pytest-cov==7.1.0 mutmut==3.8.0
mkdir -p ex1/src ex1/tests
echo "mutants/" >> .gitignore
```

ex1/pyproject.toml (o mesmo em cada exercício)

```
[tool.mutmut]
source_paths = ["src/"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
```

Dicas: Comandos do mutmut

Código

Terminal, dentro da pasta do exercício

```
pytest -q --cov=src --cov-branch --cov-report=term-missing
mutmut run # gera e testa os mutantes
mutmut results # lista os vivos
mutmut results --all true # lista todos, com o estado
mutmut show NOME # mostra a mudanca de um mutante
mutmut browse # navega pelos resultados
```

- A suíte precisa passar antes; se falhar, o mutmut para com `failed to collect stats`
- Escore: MS = mortos/(gerados − equivalentes)

Medindo e Melhorando uma Suíte



Situação do Aluno

Código do Exercício 1

ex1/src/classificacao.py

```python
def classificar_aluno(media, presenca):
    if media < 0 or media > 10:
        raise ValueError("Media invalida")
    if presenca < 0 or presenca > 100:
        raise ValueError("Presenca invalida")
    if presenca < 75:
        return "Reprovado por falta"
    if media >= 7.0:
        return "Aprovado"
    elif media >= 5.0:
        return "Recuperacao"
    else:
        return "Reprovado"
```

Situação do Aluno

Exercício 1

Suíte inicial, em `ex1/tests/test_classificacao.py`, com dois testes:

- `classificar_aluno(8.0, 80) == "Aprovado"`
- `classificar_aluno(3.0, 90) == "Reprovado"`

a) Meça a cobertura de ramos e rode o mutmut. Quantos mutantes foram gerados, mortos e vivos?

b) Agrupe os vivos pela causa e diga quais estão em linhas que a suíte nem executa

c) Acrescente testes até matar todos os vivos. Algum é equivalente? Qual o MS final?

d) Troque a conferência da mensagem por `pytest.raises(ValueError, match=msg)`. O que muda?

Cobertura Alta, Escore Baixo



Desconto Progressivo

Código do Exercício 2

Especificação: do quarto item em diante, cada item tem 10% de desconto. O total é arredondado a duas casas.

ex2/src/carrinho.py

```python
def total_com_desconto(precos):
    total = 0
    for i, preco in enumerate(precos):
        if i >= 3:
            total += preco * 0.9
        else:
            total += preco
    return round(total, 2)
```

Desconto Progressivo

Exercício 2

Suíte inicial: um teste, `assert total_com_desconto([10, 20, 30, 40, 50]) > 0`.

a) Meça a cobertura de ramos e o MS. O que os dois números dizem sobre a suíte?

b) Troque só o `assert` pelo valor exato, sem mudar a entrada. Quantos vivos restam?

c) Para cada vivo que restou, diga qual condição falhou: alcançar, infectar, propagar ou revelar

d) Acrescente as entradas que faltam até o MS chegar a 100%. Que tipo de preço foi preciso?

Mutantes Equivalentes



Temperaturas da Semana

Código do Exercício 3, parte 1

Especificação: `maior` devolve a maior temperatura da lista.

ex3/src/temperaturas.py

```python
def maior(temps):
    if not temps:
        raise ValueError("lista vazia")
    m = temps[0]
    for t in temps[1:]:
        if t > m:
            m = t
    return m
```

Temperaturas da Semana

Código do Exercício 3, parte 2

Especificação: `dia_do_maior` devolve a posição do primeiro dia com a maior temperatura.

ex3/src/temperaturas.py (continuação)

```
1  def dia_do_maior(temps):
2      if not temps:
3          raise ValueError("lista vazia")
4      pos = 0
5      for i in range(1, len(temps)):
6          if temps[i] > temps[pos]:
7              pos = i
8      return pos
```

Temperaturas da Semana

Exercício 3

Suíte inicial, para as duas funções:

- listas `[21, 25, 19, 30, 22]`, `[18]` e `[30, 21]`, com o valor exato
- lista vazia com `pytest.raises(ValueError)`, sem conferir a mensagem

a) Implemente a suíte e rode o mutmut. Liste os vivos com o `mutmut show`

b) Classifique cada vivo como equivalente ou não, com justificativa

c) Mate os não equivalentes e calcule o MS antes e depois

d) Explique por que > virar >= é equivalente numa função e não na outra

Da Especificação ao Escore Máximo



Parcelamento

Código do Exercício 4

- De 1 a 12 parcelas; fora disso, `ValueError("parcelas invalidas")`
- Até 3 parcelas, sem juros. Acima, 2% de juros por parcela além da terceira
- A parcela é arredondada a duas casas

ex4/src/parcelamento.py

```python
def parcela(valor, n):
    if n < 1 or n > 12:
        raise ValueError("parcelas invalidas")
    if n <= 3:
        juros = 0
    else:
        juros = 0.02 * (n - 3)
    total = valor * (1 + juros)
    return round(total / n, 2)
```

Parcelamento

Exercício 4

a) Antes de rodar qualquer ferramenta, derive a suíte por valor-limite (aula 6) e implemente
b) Meça a cobertura de ramos e o MS dessa suíte
c) Analise cada vivo: mate-o ou justifique que é equivalente. Qual o MS final?
d) Combine à mão dois mutantes de primeira ordem que sua suíte mata, num de segunda ordem
e) A suíte mata o mutante de segunda ordem? Relacione com o efeito de acoplamento e com Offutt (1992)

Síntese

- 100% de ramos conviveu com MS de 25%: cobertura não mede a verificação
- Primeiro o `assert` exato, depois as entradas de fronteira: cada passo mata um tipo de vivo
- Equivalência depende do que a função devolve e do que a especificação exige
- Valor-limite já mata a maior parte dos mutantes; a mutação mostra o que ficou de fora

Simulação e Teste de Software

Laboratório 9: Testes de Mutação

Prof. Luciano Rossi

Departamento de Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026